Biweekly Assessment BW03 — Weeks 5–6

Sessions covered: S21–30
Topics: Static methods, parameters, `return`, overloading, pass-by-value; class / object / `new`, constructors, `this`, instance vs static, encapsulation (`private`, getters)

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: Static methods **and** one extra class with a constructor are OK. **No `extends`. No `ArrayList`. No `throw` / `try` / `catch`.** Validate with `print` + `return`.

A. Multiple Choice (12)

Circle the best option.

1. A **method** is:
   a) A named block you can call
   b) Always a constructor
   c) Only `main`
   d) A MySQL table
2. Overloading means:
   a) Same name, different parameter lists
   b) Same name, different return type only
   c) Two classes with the same file name
   d) Using `extends`
3. `public static void main(String[] args)` is:
   a) Optional in every `.java` file
   b) The JVM entry point
   c) A constructor
   d) Only used with Spring
4. `new Student(101, "Anita")` does what first?
   a) Deletes the class
   b) Creates an object and runs the matching constructor
   c) Compiles MySQL
   d) Calls `toString` only
5. A constructor's name:
   a) Must match the class name
   b) Must be `init`
   c) Must be `main`
   d) Must start with `get`
6. `this.name = name;` uses `this` to mean:
   a) The parent class
   b) A static field only
   c) The **current object**
   d) The JVM

7. If you write any constructor, Java's default no-arg constructor:
   a) Stays forever
   b) Is no longer provided
   c) Becomes `private`
   d) Turns into `main`
8. An **instance** field is:
   a) Shared by all objects
   b) Unique per object
   c) Only allowed in `main`
   d) Always `final`
9. A **static** field is:
   a) Unique per object
   b) Illegal in Java
   c) One shared copy for the class
   d) Only for `String`
10. `private double balance;` means another class:
    a) Can assign `acct.balance = -1`
    b) **Cannot** compile `acct.balance` — must use methods
    c) Must use Spring
    d) Can only read it on Sundays
11. Why avoid `public void setBalance(double b) { this.balance = b; }` on a bank account?
    a) Getters are illegal
    b) It lets callers skip `deposit` / `withdraw` and smash the rule
    c) `private` fields cannot have methods
    d) Constructors cannot run
12. `Student s2 = s1;` (same class type) means:
    a) A clone of every field is always made
    b) Two names point at the **same** object
    c) `s1` becomes `null`
    d) Inheritance starts

B. Coding (2)

1. Write a class `LineItem` with fields `name` (String) and `price` (double), a constructor, and instance method `lineTotal(int qty)` that returns `price * qty`. Add a **static** method `withGst(double amount)` that returns `amount * 1.05`. In `main`, construct one item `"Tea"` at `15`, print `lineTotal(2)` and `withGst` of that result. No `extends`. No `throw`.

```
// your code
```

2. Write **SecureBank** `BankAccount` with **private** `holder` and `balance`, constructor `(String, double)`, `deposit`, `withdraw`, `getBalance`. Reject `amount <= 0` and overdraft with a printed message (`print` + `return`). In `main`: opening `1000`, deposit `200`, withdraw `100`, withdraw `5000`, print the balance. No `setBalance`. No `extends`. No `throw`.

```
// your code
```

C. Short answer (1)

In 3–6 sentences, explain **encapsulation** using a campus bank-account example. Why is `private` balance + `deposit` / `withdraw` safer than a public `balance` field?

(Write 3–6 sentences.)